arXiv:2306.08055v1 [cs.LG] 13 Jun 2023

Tune As You Scale: Hyperparameter Optimization For Compute Efficient Training

Abraham J. Fetterman [1] **Ellie Kitanidis** [1] **Joshua Albrecht** [1] **Zachary Polizzi** [1] **Bryden Fogelman** [1]
Maksis Knutins [1] **Bartosz Wróblewski** [1] **James B. Simon** [1 2] **Kanjun Qiu** [1]

Abstract

Hyperparameter tuning of deep learning models can lead to order-of-magnitude performance gains for the same amount of compute. Despite this, systematic tuning is uncommon, particularly for large models, which are expensive to evaluate and tend to have many hyperparameters, necessitating difficult judgment calls about tradeoffs, budgets, and search bounds. To address these issues and propose a practical method for robustly tuning large models, we present **C**ost-**A**ware Pareto **R**egion **B**ayesian **S**earch (CARBS), a Bayesian optimization algorithm that performs local search around the performance-cost Pareto frontier. CARBS does well even in unbounded search spaces with many hyperparameters, learns scaling relationships so that it can tune models even as they are scaled up, and automates much of the "black magic" of tuning. Among our results, we effectively solve the entire ProcGen benchmark just by tuning a simple baseline (PPO, as provided in the original ProcGen paper). We also reproduce the model size vs. training tokens scaling result from the Chinchilla project (Hoffmann et al., 2022), while simultaneously discovering scaling laws for every other hyperparameter, via an easy automated process that uses significantly less compute and is applicable to any deep learning problem (not just language models).

1. Introduction

Deep learning requires hyperparameter tuning, which is a black-box optimization problem with expensive evaluations. Tuning simple baselines can lead to significantly better performance for the same amount of compute (Bello et al., 2021; Hoffmann et al., 2022). A striking recent demonstration of this is the scaling laws study from DeepMind's Chinchilla project (Hoffmann et al., 2022), which showed that a 70B parameter language model can outperform a 175B model when the number of training tokens is properly tuned. Though several approaches for tuning exist, Bayesian optimization (BO) using Gaussian process (GP) surrogate models (see e.g. Shahriari et al. 2016 for a review) has emerged as the primary tool thanks to its sample efficiency, flexibility, and robustness (Snoek et al., 2012; Turner et al., 2021).

However, despite the proven value and an ever-expanding zoo of tuning algorithms, networks today are still often not exhaustively tuned. In an informal survey of researchers presented at NeurIPS 2022 (Schneider et al., 2022), approximately three-quarters of respondents reported that they tune five or fewer hyperparameters, and nearly half stated that they perform tuning manually. According to the survey, fewer than a third of researchers use more than 25 tuning trials. Aside from potentially missing out on important discoveries like the Chinchilla scaling laws, a lack of thorough tuning is concerning because it can lead to suboptimal performance, unnecessary expense, and ambiguity when comparing new methods to the previous state-of-the-art.

There are several practical limitations to current hyperparameter tuning approaches. First, as deep learning models are scaled up, evaluations become prohibitively expensive. The standard practice is to tune smaller models and then naively extrapolate to larger models, or to use the hyperparameters of the smaller models as a "warm start" and do a minimal amount of additional tuning for larger models. However, there is considerable evidence that the optimal hyperparameters depend on scale (Kaplan et al., 2020; Hoffmann et al., 2022; Yang et al., 2022). In order to efficiently tune large models, the tuner must learn these scaling relationships, exploiting statistical information gained from iterating on smaller models.

Additionally, although BO has been very successful on problems with a small number of parameters, it often performs poorly as the number of parameters grows (see e.g. Wang et al. 2013), in part because the search space grows exponentially. Global acquisition functions tend to over-explore larger search spaces since there are more highly uncertain

[1]Generally Intelligent [2]UC Berkeley. Correspondence to: Abraham J. Fetterman <abe@generallyintelligent.com>.

regions. For this reason, often only a small subset of hyperparameters are tuned in practice.

These constraints inhibit fully-automated hyperparameter tuning, as decisions about which subsets of hyperparameters to tune, how to balance performance against cost, and what scaling strategies to employ require considerable manual effort and expertise. In addition to being time-consuming, manual tuning is fundamentally a bias-prone and irreproducible way of doing science.

To overcome these limitations, we present **C**ost-**A**ware Pareto **R**egion **B**ayesian **S**earch (CARBS). CARBS is a BO algorithm that models both performance and cost (measured in training time) as GPs and jointly optimizes them. The acquisition function uses local search in the neighborhood of the current performance-cost Pareto frontier. This local search helps it optimize even in high-dimensional spaces. Furthermore, it is able to tune as it scales and can even predict scaling laws for every hyperparameter from the tuning process because it explicitly learns cost-dependent relationships. As such, it is entirely automated and does not require specifying search bounds or model training budgets. This also allows it to tune parameters such as the number of training epochs or tokens, which are usually held constant in the tuning process even though, as Hoffmann et al. 2022 proved, co-tuning these hyperparameters can dramatically improve both the performance and compute efficiency of models.

We empirically demonstrate that on challenging open problems such as ProcGen (Cobbe et al., 2019), running CARBS on one of the simplest baselines, Proximal Policy Optimization (PPO; Schulman et al. 2017), leads to effectively solving the entire benchmark. Furthermore, we reproduce the scaling laws of Hoffmann et al. 2022 by simply running CARBS to tune GPT (Brown et al., 2020) on C4 (Raffel et al., 2019). Even on well-tuned smaller problems, CARBS achieves similar performance significantly faster and more consistently.

2. Related Work

Local search. The notion of local stochastic search with an iteratively updated search distribution is common in evolutionary strategies such as Covariance Matrix Adaptation (CMA-ES; Hansen & Ostermeier 2001), though such methods typically do not learn a surrogate model of the objective function. A few variants of BO with local search also exist, e.g. using a collection of local models in rectangular trust regions (Eriksson et al., 2019) or restricting to samples the GP is certain about (Fröhlich et al., 2021). To the best of our knowledge, our work appears to be novel in its application of local search to the Pareto front of past observations.

Cost-aware BO. The works most related to ours leverage some degree of cost-awareness in the search strategy. Abdolshah et al. 2019 explicitly searches for cheap solutions first and requires users to input an ordering of the dimensions of the search space based on prior knowledge of their relative expense. Several works (Snoek et al., 2012; Swersky et al., 2013; Poloczek et al., 2016; Wu et al., 2019; Lee et al., 2020) use a canonical acquisition function such as the Expected Improvement (EI) divided by a heterogeneous cost metric. However, these methods often show poor overall performance when the best hyperparameters are also the most expensive, as they tend to oversample cheap candidates. To reduce this effect, Lee et al. 2020 implements a cost-cooling trick that transitions from cost-normalized EI to plain EI, though this requires knowing the compute budget a priori. Guinet et al. 2020 minimizes cost constrained by EI, using a subset of candidates with sufficiently high EI at each step. Lee et al. 2021 and Astudillo et al. 2021 formulate the problem as a Markov decision process where a "non-myopic" acquisition function uses rollouts from the surrogate. These methods focus on finding the cheapest solutions that still perform well or the best performing methods within a given compute budget rather than on learning the structure of the Pareto front itself.

Neural scaling laws. A body of recent work focuses on the empirical scaling relationships between a network's performance, size, compute budget, and the amount of training data it sees. Influential early works (Hestness et al., 2017; Rosenfeld et al., 2019; Tan & Le, 2019; Li et al., 2020; Kaplan et al., 2020) laid the groundwork and a number of subsequent works have explored scaling laws across several modalities, architectures, and problem settings (e.g. Tay et al. 2021; Zhai et al. 2021; Henighan et al. 2020; Droppo & Elibol 2021; Cherti et al. 2022). A notable reminder of the importance of accurate neural scaling laws is Hoffmann et al. 2022, which demonstrated superior performance to its compute-equivalent precursor Gopher with a model that was 4x smaller. Scaling studies typically involve training large families of models across wide ranges of dataset sizes and compute budgets, and then analytically fitting the results. To draw reliable conclusions, each evaluation should ideally be well-tuned in the other hyperparameters, further adding to the computational burden. In practice, most studies are restricted to systematically varying a few dimensions at a time while holding the others fixed, which can miss improvements from correlated changes to several variables.

3. Algorithm

CARBS is a cost-aware optimization algorithm that is local to the compute-efficient Pareto front. We first give a simplified overview of the algorithm, then add some finer details in the following sections.

3.1. Simplified algorithm

We are interested in maximizing a function f of real input parameters $\mathbf{x}$ that we observe through a noisy channel $y = f(\mathbf{x}) + \epsilon_y$ where ϵ_y is a mean zero Gaussian. In addition to the output of the function, we observe the cost c of evaluating the function. We assume without loss of generality that our input parameters and outputs $\mathbf{x}$ and y are scaled to be of order one.

The role of this algorithm is to suggest parameters $\mathbf{x}'$ that will maximize $f(\mathbf{x}')$, ignoring, for now, other heuristics related to the cost of evaluation. At a high level, CARBS works by first generating candidates from a local search space, evaluating those candidates with GP surrogates, then selecting the highest scoring candidate as scored by an acquisition function based on those surrogates. Each of these steps is explained more precisely below.

Generating candidates in the local search space

CARBS defines the local search space around points on the observed cost-performance Pareto front. If we assume we have t observations, and that observation i has parameters $\mathbf{x}_i$, output y_i and cost c_i, then the Pareto front is

$$\text{PF} = \{i \in [1:t] \text{ s.t. } \{y_i > y_j \vee c_i < c_j\} \, \forall j \neq i\}. \quad (1)$$

We define the search space as a set of Gaussian distributions of radius σ_{search} around these parameters, defining the unnormalized probability density of $\mathbf{x}$ as

$$P_{\text{search}}(\mathbf{x}) = \max_{i \in \text{PF}} \left[\exp\left(-\frac{|\mathbf{x}_i - \mathbf{x}|^2}{2\sigma^2_{\text{search}}} \right) \right]. \quad (2)$$

Candidates for the next suggestion are sampled uniformly from the Gaussian distributions $\mathcal{N}(\mathbf{x}_i, \sigma_{\text{search}})$. From an evolutionary strategies perspective, these can be described as mutations from a dynamic pool of parents (the Pareto set).

Evaluating candidates via Gaussian process surrogates

Once we have the candidate suggestions, we use GP surrogates to estimate the cost $\tilde{c}$ and output $\tilde{y}$ based on the parameters $\mathbf{x}$ of each candidate. CARBS uses three GPs: $\mathcal{GP}_y$ to predict the performance of a candidate, informed by all observations so far; $\mathcal{GP}_c$ to predict the cost of a candidate, informed by all observations so far; and $\mathcal{GP}_{\text{pf}}$ to predict the optimal performance corresponding to that cost, informed only by the Pareto front so far. This last GP can be thought of as modeling the shape of the Pareto front and is used to calculate the baseline against which performance is compared in the EI acquisition function.

For these surrogates, we utilize the same kernel function used in Heteroskedastic Evolutionary Bayesian Optimization (HEBO; Cowen-Rivers et al. 2020), the sum of a linear and Matern kernel. We find that the inclusion of the linear kernel in particular is crucial for getting good performance, as without it the model extrapolates poorly. The third GP, which estimates the Pareto front based only on the cost, uses an RBF kernel. In summary, we fit the GP models using the Gaussian process likelihood p and kernels k_θ,

$$\begin{aligned}
\mathcal{GP}_y &\leftarrow \max_\theta \left[p\left(y | \{\mathbf{x}_i, y_i\}_{i \in [1:t]}, k_\theta = k_{\text{lin}} + k_{\text{Mat}}\right) \right], \\
\mathcal{GP}_c &\leftarrow \max_\theta \left[p\left(c | \{\mathbf{x}_i, c_i\}_{i \in [1:t]}, k_\theta = k_{\text{lin}} + k_{\text{Mat}}\right) \right], \\
\mathcal{GP}_{\text{pf}} &\leftarrow \max_\theta \left[p\left(y | \{c_i, y_i\}_{i \in PF}, k_\theta = k_{\text{RBF}}\right) \right].
\end{aligned} \quad (3)$$

In the following notation, we use $\mathcal{GP}_y(\mathbf{x})$ to indicate the posterior distribution of $\mathcal{GP}_y$ evaluated at $\mathbf{x}$.

Scoring candidates via the acquisition function

The final model $\mathcal{GP}_{\text{pf}}$ is fit only using observations that belong in the Pareto front, and is used as the baseline for the expected improvement (EI) in the acquisition function in composition with the cost model. Thus, the acquisition function explicitly seeks to improve performance relative to the corresponding point on the Pareto front. That is, with the expected cost $\tilde{c}(\mathbf{x}) = \mathbb{E}[\mathcal{GP}_c(\mathbf{x})]$ we can use the Pareto value $\tilde{y}_{\text{pf}}(\mathbf{x}) = \mathbb{E}[\mathcal{GP}_{\text{pf}}(\tilde{c}(\mathbf{x}))]$ to write the expected improvement with respect to the Pareto front,

$$\alpha_{\text{EI-pf}}(\mathbf{x}) = \mathop{\mathbb{E}}_{\mathcal{GP}_y} \left[\text{ReLU}\left(\mathcal{GP}_y(\mathbf{x}) - \tilde{y}_{\text{pf}}(\mathbf{x})\right) \right], \quad (4)$$

where ReLU is the rectified linear unit and the outer expectation is over the distribution output by $\mathcal{GP}_y$. We combine this acquisition function with the search space probability naively by taking the product, and the final suggestion is simply the max over all candidates:

$$\alpha_{\text{pf}}(\mathbf{x}) = \alpha_{\text{EI-pf}}(\mathbf{x}) \, P_{\text{search}}(\mathbf{x}). \quad (5)$$

This effectively adds a soft trust region to the EI acquisition function. A hard trust region would discard any solution greater than some distance from a PF point, which could be implemented by using a uniform probability distribution within a sphere of radius σ for P_{search}. Instead of a hard cutoff, we discount points further from the trust region by using a Gaussian probability distribution.

The above acquisition function is enough to define a simplified version of CARBS found in the Appendix as Algorithm 1. The next section fills in details that improve performance on practical problems.

3.2. Additional Details

Acquisition function clamping. An issue with the acquisition function $\alpha_{\mathrm{EI-pf}}$ is that it uniformly weights improvements across the entire Pareto front, while we are more interested in the best performing parameters. The regular expected improvement uses $y_{\max} = \max(y_i)$,

$$\alpha_{\mathrm{EI-max}}(\mathbf{x}) = \mathop{\mathbb{E}}_{\mathcal{GP}_y}\left[\mathrm{ReLU}\left(\mathcal{GP}_y(\mathbf{x}) - y_{\max}\right)\right]. \quad (6)$$

We seek a balance between these two functions: empirically, $\alpha_{\mathrm{EI-max}}$ only samples near the highest performing part of the Pareto front, and $\alpha_{\mathrm{EI-pf}}$ only rarely samples there. Our solution is to sample a threshold cost $c_{\mathrm{th}} \sim \mathrm{LogUniform}\left(\min_{PF}(c_i), \max_{PF}(c_i)\right)$ from a log uniform distribution from the minimum Pareto front cost to the maximum Pareto front cost. We then evaluate the Pareto front performance there, $\tilde{y}_{\mathrm{th}} = \mathbb{E}\left[\mathcal{GP}_{\mathrm{pf}}(c_{\mathrm{th}})\right]$,

$$\alpha_{\mathrm{EI-th}}(\mathbf{x}) = \mathop{\mathbb{E}}_{\mathcal{GP}_y}\left[\mathrm{ReLU}\left(\mathcal{GP}_y(\mathbf{x}) - \max\left(\tilde{y}_{\mathrm{pf}}(\mathbf{x}), \tilde{y}_{\mathrm{th}}\right)\right)\right]. \quad (7)$$

See experiments comparing $\alpha_{\mathrm{EI-th}}$ with α_{EI} and $\alpha_{\mathrm{EI-pf}}$ in Section 4.3.4. In all other experiments, $\alpha_{\mathrm{EI-th}}$ is used.

Output warping. The GP models depend on the output variable having an approximately Gaussian distribution with zero mean and unit variance. HEBO showed that using a Box-Cox or Yeo-Johnston transform could improve performance of their Bayesian optimization algorithm (Cowen-Rivers et al., 2020). We use a similar, parameter free method, the `scikit-learn` (Pedregosa et al., 2011) `QuantileTransform`[1] with $\sqrt{t}$ bins, to transform the output value y. We find the use of the parameter free transform to be more stable and repeatable than the learned parameter transforms.

Resampling. Noisy observations are particularly important to handle correctly for CARBS, as the best observations are used to define the local search space and thus a particularly extreme noise point might prevent the algorithm from finding a better optimum. To mitigate this issue, points on the Pareto front are resampled, or suggested again, at a fixed rate: one out of every N_{resample} suggestions is a set of parameters that has already been observed. Specifically, we count the samples for each of the existing Pareto front points and suggest the parameters which have the smallest number of samples, breaking ties by choosing the one with the lowest cost.

This resampling also allows us to more carefully select our Pareto set. We first group all observations by parameters, and then calculate the mean output $\bar{y}$, max output y^+, average cost $\bar{c}$ and number of samples n for each group. We then define the grouped Pareto set as

$$PF_{\mathrm{group}} = \{i \in [1:g] \text{ s.t. } \begin{cases} y_i^+ > y_j^+ \vee \bar{c}_i < \bar{c}_j & (n_i = 1) \\ \bar{y}_i > \bar{y}_j \vee \bar{c}_i < \bar{c}_j & (n_i > 1) \end{cases} \forall j \neq i\}. \quad (8)$$

This includes single observations in the Pareto front only if they are better than the best observation that is lower cost, but allows resampled groups if their mean is higher than the best group mean that is lower cost.

Pareto front minimum. Because the cost itself is a noisy parameter, the very lowest cost observation is usually not very interesting to include in our Pareto front. We start the Pareto front with best performing observation among the 20% of observations with the lowest cost.

Failure prediction. Tuning parameters like model size can be difficult if some parameters lead to failure (e.g., from out-of-memory errors). Including these as observations with poor results (for example, setting $y_i = 0$) results in GP models that are not accurate near failure boundaries. We instead train a separate GP regression model to predict whether training will fail, setting $f_i = 1$ if the result of a run is a failure or $f_i = -1$ if it is a success,

$$\mathcal{GP}_{\mathrm{fail}} \leftarrow \max_{\theta}[p(f \mid \{\mathbf{x}_i, f_i\}_{i\in[1:t]}, k_\theta = k_{\mathrm{lin}} + k_{\mathrm{Mat}})]. \quad (9)$$

We then calculate the probability of success as

$$P_{\mathrm{success}}(\mathbf{x}) = \mathop{\mathbb{E}}_{\mathcal{GP}_{\mathrm{fail}}}\left[\mathcal{H}\left(-\mathcal{GP}_{\mathrm{fail}}(\mathbf{x})\right)\right], \quad (10)$$

where $\mathcal{H}$ is the Heaviside function. We add this to our acquisition function to get

$$\alpha_{\mathrm{CARBS}}(\mathbf{x}) = \alpha_{\mathrm{EI-th}}(\mathbf{x}) P_{\mathrm{search}}(\mathbf{x}) P_{\mathrm{success}}(\mathbf{x}).$$

Cost ceiling. We add a cost maximum so that CARBS will not suggest parameters that take too long to evaluate. We implement this with a hard threshold by removing suggestions where $\tilde{c} > c_{max}$.

Parallelism. We would like to have multiple different suggestions under evaluation in parallel. The above algorithm only depends on the past observations, and so it is likely

[1] QuantileTransform achieves the same goal as the PowerTransform used by HEBO but is more numerically stable.

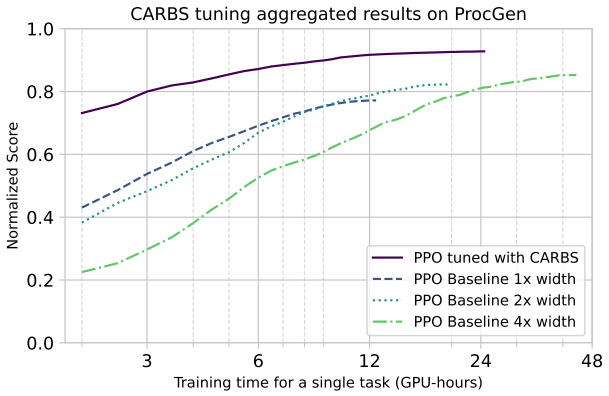

Figure 1. Aggregate normalized score vs. training time per task. The solid purple curve is the Pareto front of the CARBS-tuned PPO while the dashed lines are the learning curves of the baselines (the previous state-of-the-art performance on ProcGen). ProcGen is usually tuned for 200M steps, corresponding to 12 hours in our setup. CARBS tuning can improve performance by >16% in the same amount of training time or match it in 4x less time.

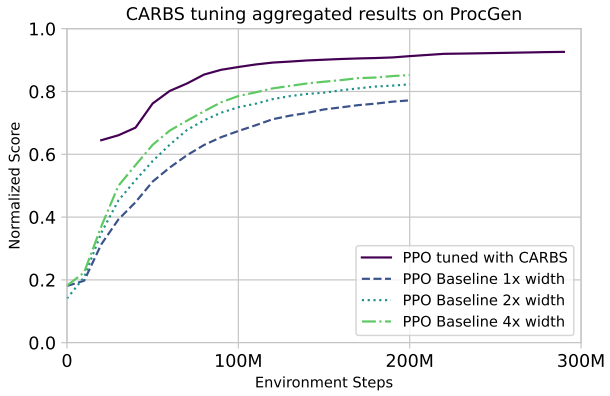

Figure 2. Aggregate normalized score vs. environment steps per task. The solid purple curve is the Pareto front of the CARBS-tuned PPO while the dashed lines are the learning curves of the baselines (the previous state-of-the-art performance on ProcGen). ProcGen is usually tuned for 200M steps. CARBS tuning can match the state-of-the-art performance with 1.5-4x higher sample efficiency.

to make similar suggestions until new data is obtained. To avoid this, we use Thompson sampling to sample from the GP posterior, which we then use to obtain predicted outputs $\hat{y}_i$ from $\mathcal{GP}_y$ for the outstanding suggestions $\mathbf{x}_i$ (parameters that have been suggested but not observed). We then train the output model $\mathcal{GP}_y$ on these predicted outputs together with the observations.

4. Experiments

4.1. Solving the ProcGen Benchmark with PPO

Reinforcement learning algorithms are notoriously difficult to tune as they have many hyperparameters, including cost-influencing parameters such as the number of parallel environments, the number of passes over the data, and the network width. Further complicating tuning, reinforcement learning environments can be very noisy.

To test whether CARBS can consistently improve performance in this type of environment, we tune 13 hyperparameters (see Appendix Table 1) of a PPO baseline on each of the 16 tasks of the ProcGen benchmark. We find that CARBS dramatically improves both performance and training cost on all tasks that are not already solved by the baseline, in many cases fully solving them. For the three tasks that are already solved by the baseline, CARBS still significantly reduces the time required to solve the task. To the best of our knowledge, this CARBS-tuned PPO beats all other state-of-the-art results on ProcGen.

In detail: The results from Cobbe et al. 2019 are still the best-performing published PPO hyperparameters, so we use that as our baseline. We also include the 2x and 4x width configurations as additional baselines. We stress that these baselines were tuned by the original authors, just not with CARBS. We normalize the per-task scores by dividing by the maximum theoretical score on each task, although a perfect score is not necessarily achievable even by humans.

Figure 1 shows the Pareto front found by CARBS compared with the learning curves of the baselines. The ProcGen benchmark is typically trained for 200M environment steps. Our setup with one NVIDIA 3090 GPU and 8 CPU cores is able to train the 1x width baseline in about 12 hours per task and achieves an aggregate normalized score of 0.79. After tuning each task with CARBS with 8 parallel workers for 9 days, the aggregate performance in 12 hours per task improves to 0.92, an increase of more than 16%. In addition to better performance at 12 hours, CARBS finds a configuration that can get to the same 0.79 performance 4x faster. Results for individual tasks are shown in Appendix Figure 7.

Although we did not explicitly search for algorithms with improved sample efficiency, we are able to use the observations from our tuning results to construct another Pareto curve trading-off score with number of environment steps, shown in Figure 2. These results demonstrate that CARBS-tuned PPO can get to the same performance with 1.5-4x less data than the baseline. Results for individual tasks are shown in Appendix Figure 8.

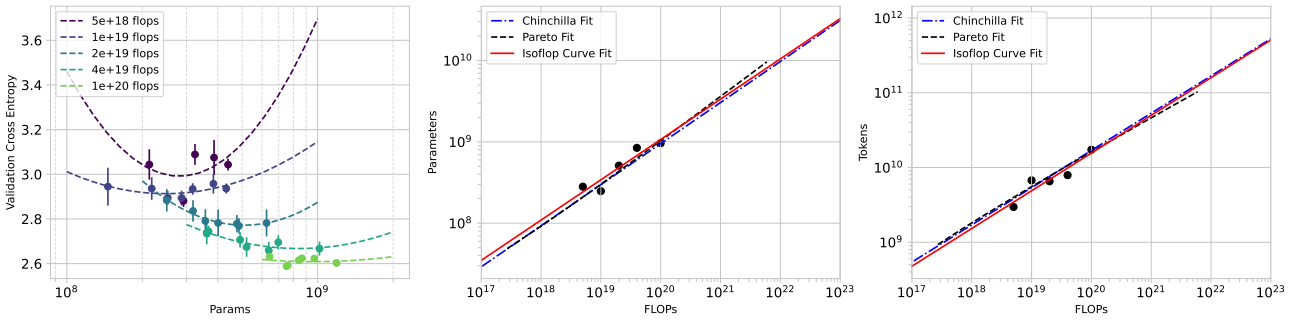

Figure 3. Left: Approximate isoflop curves for GPT-like Transformers trained on C4. Middle and Right: Scaling predictions for optimal parameters and tokens based on fit isoflop curves (blue circles and line) and a linear fit to the Pareto front (black dashed lines).

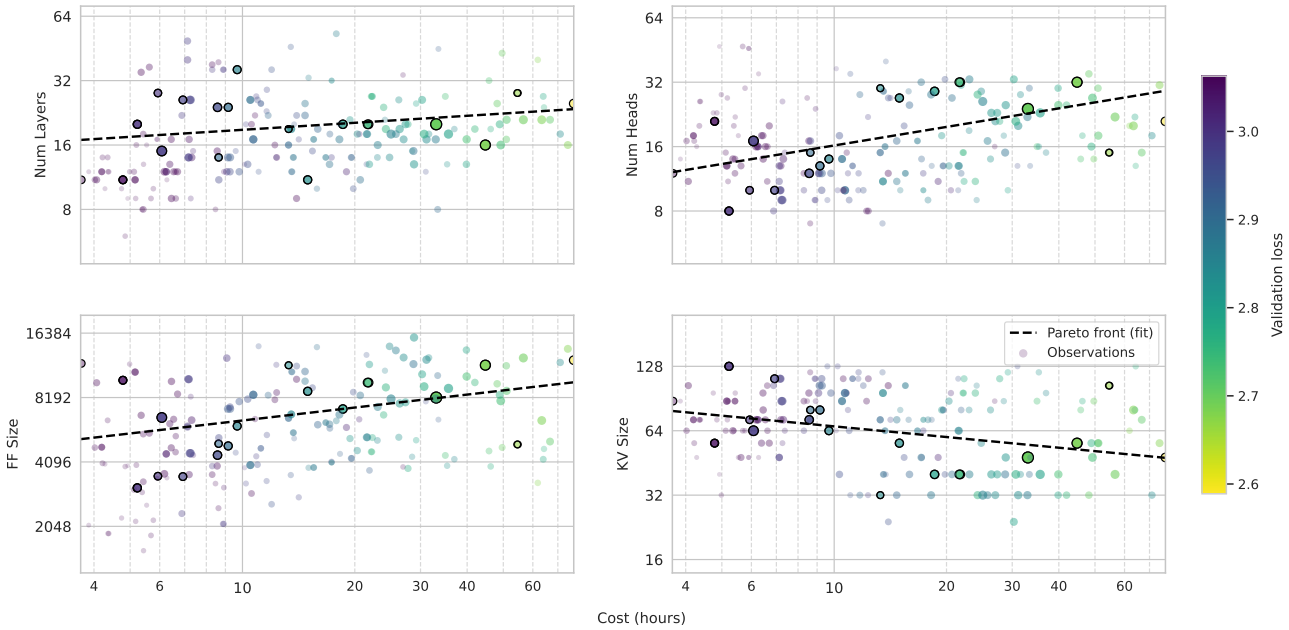

Figure 4. Fit scaling predictions for different model size parameters from a CARBS run on GPT-like Transformers trained on C4. The color of the observations reflects the final validation loss of each run on the C4 dataset. The size and transparency of each data point reflect the distance to the fit Pareto front (smaller and lighter means further away). The observations with the black border are part of the Pareto front.

4.2. Scaling laws for language modeling

With enough data, the GP model CARBS uses can automatically discover scaling laws and leverage them to improve performance. We test the discovery of scaling laws with CARBS by tuning GPT-like Transformers to minimize validation loss on the C4 dataset, and compare our predicted scaling laws with those reported in the Chinchilla paper (Hoffmann et al., 2022). We find that the predictions are very similar, with a detailed investigation matching the Chinchilla results exactly.

We used a Transformer implementation from Mosaic ML (MosaicML, 2021), together with some implementation details to better match Chinchilla: we use the SentencePiece tokenizer `t5-small` (Raffel et al., 2020), use relative positional encodings (Shaw et al., 2018), and set the initialization to be the same as T5 (Raffel et al., 2020). We also used a few additional parameters for initialization we attribute to the muP scaling (Yang et al., 2022): separate initialization scale for the embedding and Transformer, as well as scale parameters we tuned on the attention softmax and final output. Together, we tuned 19 different parameters including

regularization terms, schedule, model size and token count.

We ran CARBS with 8 workers in parallel for 35 days, setting CARBS to suggest only parameters which would take less than 5 days of training time. Each worker was a single machine with 8x 40GB A100 GPUs. We started tuning at a model size of 125M parameters.

We set out to replicate *Approach 2* proposed in the Chinchilla paper by taking isoflop slices from our pool of observations. Figure 3 shows approximate isoflop curves that are derived from the data using a method described in Appendix C.1. A linear fit to the minima of each of these isoflop curves yields $a = 0.50$ for $N_{opt} \propto C^a$ and $b = 0.50$ for $D_{opt} \propto C^b$. The Chinchilla results for a and b were also 0.50 (see their Table A2), indicating exact agreement with our findings despite the noisy data.

We used a simplified method to predict scaling laws for all of the parameters being searched over. We simply fit a linear regression model to all of the parameters of the search space in their transformed space, using only the Pareto front observations as input. This model is shown in Fig 3 and 4 as a black dashed line. One can see that it is only slightly divergent from the previous prediction in Figure 3, with a slope around 0.51 to 0.55 for a and 0.45 to 0.49 for b.

We show how this model fits different terms related to the model size in Figure 4. Although the data is quite noisy, we can see that the best fit is to not just uniformly increase the model size. The parameter most sensitive to scale is the width of the feed-forward MLPs, with the network depth increasing more slowly. More intriguingly, the model dimension $d_{model} = d_{kv} \cdot n_{head}$ is held almost constant, with a *decrease* in the size of the attention q, k, v dimensions at larger scale. We include this type of plot for every search parameter in Appendix C.2.

For our analysis, CARBS observed 340 training runs, with models from 19M to 1.5B parameters trained for 600M to 24B tokens. By comparison, Chinchilla's analysis required training 50 models ranging from 44M to 16B parameters (see their Table A9) with 9 different learning rate schedules/number of training tokens and held all other hyperparameters fixed. We thus conclude that the CARBS analysis required less compute despite also marginalizing over many other hyperparameters and obtaining scaling relationships for each of them as well.

4.3. Comparison with other tuners

CARBS provides a means to thoroughly and robustly tune large, complex, noisy models with non-trivial hyperparameter search spaces, of the sort that are historically difficult to tune yet are of increasing research interest. This makes comparisons to existing tuners challenging. Nevertheless, for completeness, we provide a comparison to four existing algorithms on smaller problems that these methods are better suited to handle. We emphasize that the main goal of CARBS is not to move the needle on these types of problems but rather to enable tuning on problems that are intractable with current methods.

We design three smaller tuning problems across different domains: language modeling, reinforcement learning, and image classification. We compare against four tuners, chosen for their popularity, existing implementations in the Ray Tune library (Liaw et al., 2018), and diversity. The details of the tasks and baselines are given below.

4.3.1. TASKS

Language modeling. We tokenize the `WikiText103` dataset (Merity et al., 2016) using whole words, and train a masked Transformer language model (Vaswani et al., 2017) according to a PyTorch example script (Pytorch, 2022a). We use defaults primarily from the `gpt-micro` setting of `mingpt` as our initial search center (Karpathy, 2022).

Reinforcement learning. We train a PPO Schulman et al. 2017) policy on the `Ant-v4` MuJoCo gym environment (Brockman et al., 2016) using envpool for parallelism (Weng et al., 2022). We use a recent implementation of PPO that has carefully reproduced published results (Huang et al., 2022). We use default parameters from that script as our initial search center.

Image classification. We train convolutional neural network models to classify images from the imagenette dataset (FastAI, 2022). We use the PyTorch example script (Pytorch, 2022b) for training, with the default torchvision ResNet (He et al., 2016) modified to allow us to vary the depth and width. We also add random augmentations (Cubuk et al., 2020) from `torchvision` and label smoothing (Szegedy et al., 2016). We use default parameters from the original script and from the ResNet-RS paper as our initial search center (Bello et al., 2021).

We explicitly include cost-related parameters like the model size, batch size, and the amount of training data in the search space of around 10 parameters for each problem. All parameters we tuned are described in Appendix D. The tuning experiments were run with 8 independent workers with a single GPU each, and the best set of parameters was chosen after 12 hours of tuning time. This is a significantly higher compute requirement than other baselines like Bayesmark (Turner et al., 2021), which is more representative of practical problems in machine learning.

4.3.2. BASELINES

Blended search, an algorithm that utilizes local search starting at a set of specified low cost parameters (Wang et al., 2021). It combines local search with a global algorithm for

starting and stopping the local search threads.

HEBO, the winner of the 2020 NeurIPS Black Box Optimization Challenge (Cowen-Rivers et al., 2020; Turner et al., 2021). It utilizes a GP model of past observations as well as leveraging several BO acquisition functions in parallel for suggestion.

Tree of Parzen Estimators (TPE), a popular algorithm for hyperparameter tuning that maintains a tree-based model of all past observations (Akiba et al., 2019).

Asynchronous Successive Halving Algorithm (ASHA), a variant of random search that terminates the lowest performing runs early (Li et al., 2018). By doing so, it is able to sample many more parameters than other algorithms.

4.3.3. Fairness of comparison

While CARBS only requires specifying an initial search center for each parameter, the baseline algorithms require finite search bounds. We choose these bounds by running CARBS three times on each problem and choosing minimum and maximum values for each search parameter that included the observations that ran successfully (see Appendix D).

We endeavor to make the comparison as fair as possible, avoiding choices that give CARBS an advantage (e.g. by using wider default search bounds instead of the search subspace that CARBS discovers, or by adding more hyperparameters to tune). Additionally, these baseline algorithms have no way of modeling training failures, such as out-of-memory errors, so we return a constant result for failed runs corresponding to the worst observed result for that task.

4.3.4. Results

We find that CARBS performs comparably to the other algorithms on all tasks (see Figure 5), and is the best-scoring algorithm on the language modeling task.

In addition to high median performance, CARBS produces a much lower variance distribution of outputs. This robust tuning is particularly useful when comparing two tuned algorithms with each other. Similarly, we note that CARBS performance is consistent across tasks whereas e.g. ASHA is the highest performing algorithm on the reinforcement learning task but the lowest on the image classification task.

With the search spaces we have defined, it is useful to compare tuning runs not just in terms of best overall performance, but also in terms of how long it takes to achieve that performance since training time can vary by several orders of magnitude. We compare the observations and the resulting Pareto fronts for different tuning algorithms on the reinforcement learning task in Fig 6.

Compared to other hyperparameter tuning algorithms,

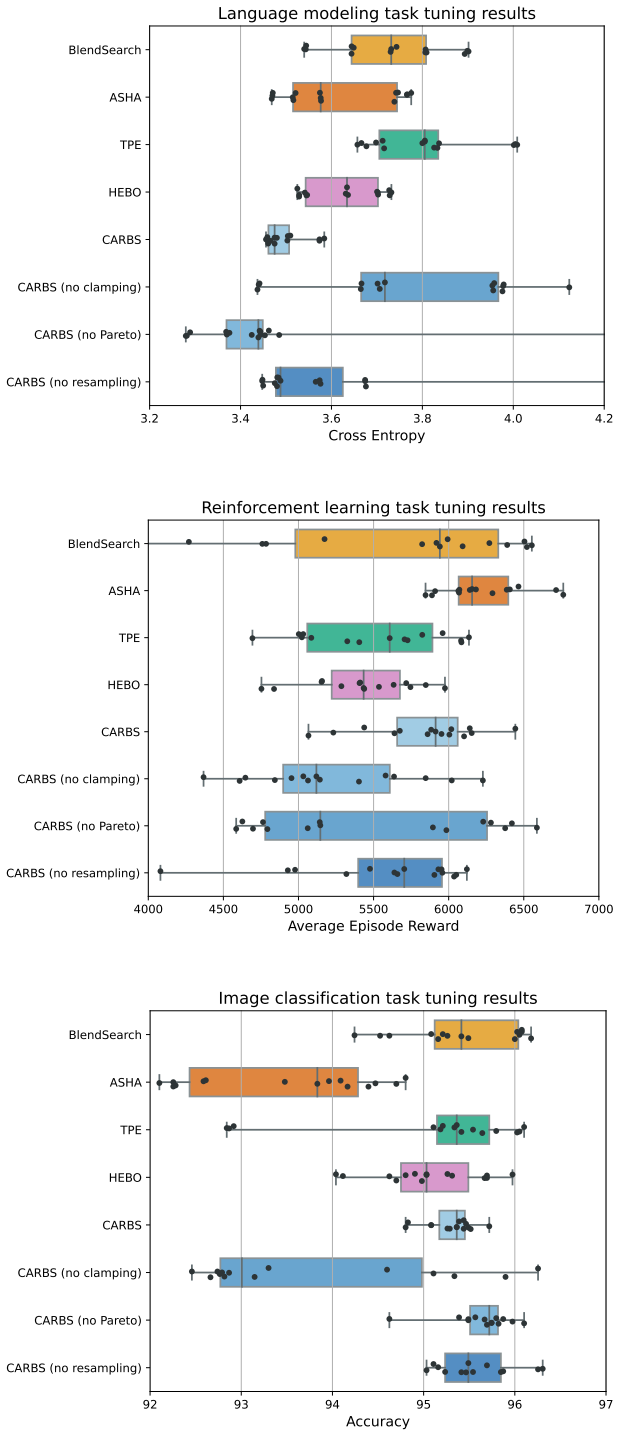

Figure 5. Final performance of different hyperparameter tuning algorithms on the language modeling task (top), reinforcement learning task (middle) and image classification task (bottom). Each algorithm was run five times and the best set of parameters was rerun three times.

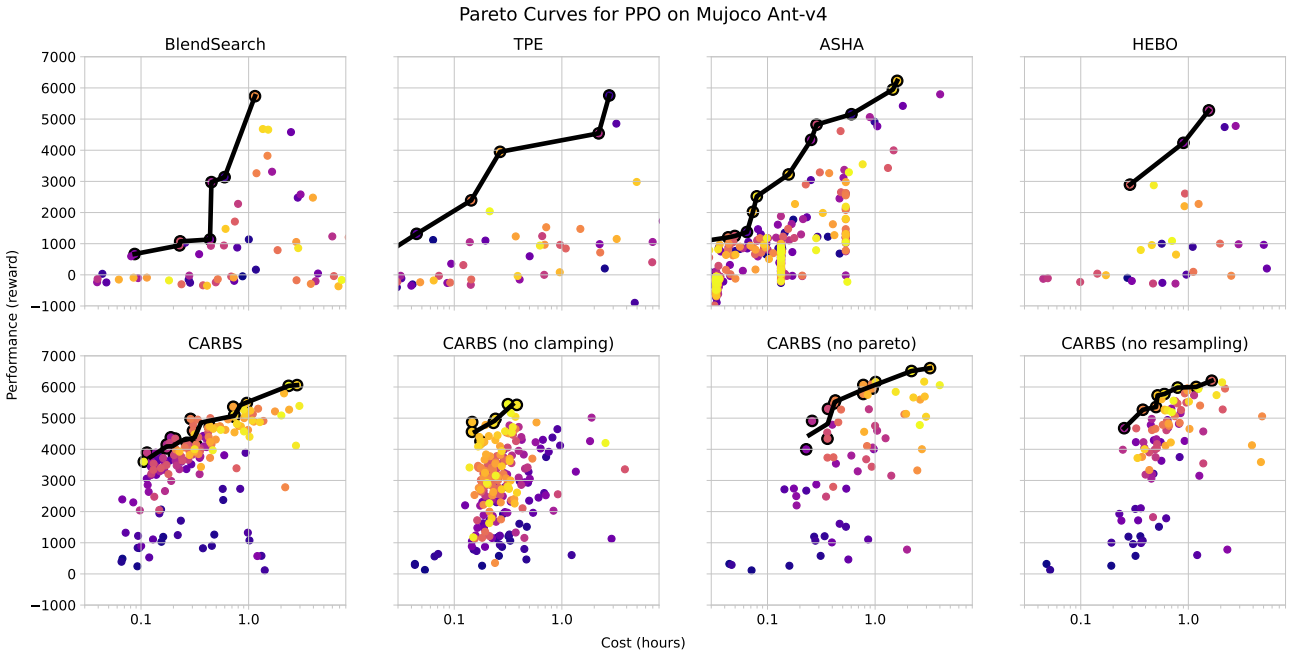

Figure 6. Observations (dots) and Pareto fronts (black lines) for a single 12 hour, 8-worker training run tuning PPO on MuJoCo `Ant-v4` for a variety of training algorithms. The color of the observations indicates the order observations were made, from purple at the start of training through magenta, orange, and yellow for the final observations. CARBS has more total observations because it uses its budget more efficiently.

CARBS produces many more samples close to the Pareto front. This is useful for producing a better surrogate model and for making predictions for how to best scale parameters. While observations far from the Pareto front narrow down the global hyperparameter space, they do not contribute to a useful model of performance near the Pareto front.

We also note the interesting failure modes of different CARBS ablations depicted in these charts. The version of CARBS that always uses the maximum observation for the EI (no Pareto) does not sample as many points close to the Pareto front, as it samples more high cost points. Although this does sometimes lead to higher performance, it is less robust than the other acquisition function, as can be seen in the aggregated box plots (Fig 5). The version that uses the acquisition function $\alpha_{\mathrm{EI-pf}}$ without the random cost clamping (no clamping) includes more low cost points at the expense of exploring higher cost, higher performing ones. And the version of CARBS without resampling (no resampling) does not discover the same cost-performance scaling relationship because of the noisy observations.

5. Conclusion

We presented CARBS, a new Bayesian optimization algorithm for hyperparameter tuning that models the performance-cost Pareto frontier. CARBS benefits from a local search strategy in the vicinity of the current Pareto set at each iteration, which enables it to efficiently optimize even when the number of hyperparameters is large, and also benefits from its cost-awareness, which helps it automatically learn how to tune as it scales. We demonstrated that simple algorithms tuned with CARBS achieve breakthrough performance, and that scaling laws can be derived from the tuning process without additional effort. Though rigorous manual scaling studies are the best way to have confidence in extrapolating to larger scales, we hope CARBS will provide an easier way to do this on a lot of problems.

References

Abdolshah, M., Shilton, A., Rana, S., Gupta, S., and Venkatesh, S. Cost-aware Multi-objective Bayesian optimisation. art. arXiv:1909.03600, September 2019. doi: 10.48550/arXiv.1909.03600.

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

Astudillo, R., Jiang, D. R., Balandat, M., Bakshy, E., and

Frazier, P. I. Multi-Step Budgeted Bayesian Optimization with Unknown Evaluation Costs. art. arXiv:2111.06537, November 2021. doi: 10.48550/arXiv.2111.06537.

Bello, I., Fedus, W., Du, X., Cubuk, E. D., Srinivas, A., Lin, T., Shlens, J., and Zoph, B. Revisiting resnets: Improved training and scaling strategies. *CoRR*, abs/2103.07579, 2021. URL https://arxiv.org/abs/2103.07579.

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym, 2016.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language Models are Few-Shot Learners. art. arXiv:2005.14165, May 2020. doi: 10.48550/arXiv.2005.14165.

Cherti, M., Beaumont, R., Wightman, R., Wortsman, M., Ilharco, G., Gordon, C., Schuhmann, C., Schmidt, L., and Jitsev, J. Reproducible scaling laws for contrastive language-image learning. art. arXiv:2212.07143, December 2022. doi: 10.48550/arXiv.2212.07143.

Cobbe, K., Hesse, C., Hilton, J., and Schulman, J. Leveraging Procedural Generation to Benchmark Reinforcement Learning. art. arXiv:1912.01588, December 2019. doi: 10.48550/arXiv.1912.01588.

Cowen-Rivers, A. I., Lyu, W., Tutunov, R., Wang, Z., Grosnit, A., Rhys Griffiths, R., Maraval, A. M., Jianye, H., Wang, J., Peters, J., and Ammar, H. B. HEBO Pushing The Limits of Sample-Efficient Hyperparameter Optimisation. art. arXiv:2012.03826, December 2020. doi: 10.48550/arXiv.2012.03826.

Cubuk, E. D., Zoph, B., Shlens, J., and Le, Q. V. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, pp. 702–703, 2020.

Droppo, J. and Elibol, O. Scaling Laws for Acoustic Models. art. arXiv:2106.09488, June 2021. doi: 10.48550/arXiv.2106.09488.

Eriksson, D., Pearce, M., Gardner, J. R., Turner, R., and Poloczek, M. Scalable global optimization via local bayesian optimization. *CoRR*, abs/1910.01739, 2019. URL http://arxiv.org/abs/1910.01739.

FastAI. imagenette. https://github.com/fastai/imagenette, 2022.

Fröhlich, L. P., Zeilinger, M. N., and Klenske, E. D. Cautious bayesian optimization for efficient and scalable policy search. In *Proceedings of the 3rd Conference on Learning for Dynamics and Control*, volume 144 of *Proceedings of Machine Learning Research*, pp. 227–240. PMLR, 07 – 08 June 2021.

Guinet, G., Perrone, V., and Archambeau, C. Pareto-efficient Acquisition Functions for Cost-Aware Bayesian Optimization. art. arXiv:2011.11456, November 2020. doi: 10.48550/arXiv.2011.11456.

Hansen, N. and Ostermeier, A. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9:159–195, 06 2001. doi: 10.1162/106365601750190398.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

Henighan, T., Kaplan, J., Katz, M., Chen, M., Hesse, C., Jackson, J., Jun, H., Brown, T. B., Dhariwal, P., Gray, S., Hallacy, C., Mann, B., Radford, A., Ramesh, A., Ryder, N., Ziegler, D. M., Schulman, J., Amodei, D., and McCandlish, S. Scaling laws for autoregressive generative modeling. *CoRR*, abs/2010.14701, 2020. URL https://arxiv.org/abs/2010.14701.

Hestness, J., Narang, S., Ardalani, N., Diamos, G. F., Jun, H., Kianinejad, H., Patwary, M. M. A., Yang, Y., and Zhou, Y. Deep learning scaling is predictable, empirically. *CoRR*, abs/1712.00409, 2017. URL http://arxiv.org/abs/1712.00409.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. Training Compute-Optimal Large Language Models. art. arXiv:2203.15556, March 2022. doi: 10.48550/arXiv.2203.15556.

Huang, S., Dossa, R. F. J., Raffin, A., Kanervisto, A., and Wang, W. The 37 implementation details of proximal policy optimization. In *ICLR Blog Track*, 2022. URL https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/. https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling Laws for Neural Language Models. art. arXiv:2001.08361, January 2020. doi: 10.48550/arXiv.2001.08361.

Karpathy, A. mingpt. https://github.com/karpathy/minGPT, 2022.

Lee, E. H., Perrone, V., Archambeau, C., and Seeger, M. Cost-aware Bayesian Optimization. art. arXiv:2003.10870, March 2020. doi: 10.48550/arXiv.2003.10870.

Lee, E. H., Eriksson, D., Perrone, V., and Seeger, M. A Nonmyopic Approach to Cost-Constrained Bayesian Optimization. art. arXiv:2106.06079, June 2021. doi: 10.48550/arXiv.2106.06079.

Li, L., Jamieson, K., Rostamizadeh, A., Gonina, E., Hardt, M., Recht, B., and Talwalkar, A. A System for Massively Parallel Hyperparameter Tuning. art. arXiv:1810.05934, October 2018. doi: 10.48550/arXiv.1810.05934.

Li, Z., Wallace, E., Shen, S., Lin, K., Keutzer, K., Klein, D., and Gonzalez, J. E. Train Large, Then Compress: Rethinking Model Size for Efficient Training and Inference of Transformers. art. arXiv:2002.11794, February 2020. doi: 10.48550/arXiv.2002.11794.

Liaw, R., Liang, E., Nishihara, R., Moritz, P., Gonzalez, J. E., and Stoica, I. Tune: A research platform for distributed model selection and training. *arXiv preprint arXiv:1807.05118*, 2018.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

MosaicML. composer. https://github.com/mosaicml/composer/, 2021.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Poloczek, M., Wang, J., and Frazier, P. I. Multi-Information Source Optimization. art. arXiv:1603.00389, March 2016. doi: 10.48550/arXiv.1603.00389.

Pytorch. Language modeling with nn.transformer and torchtext. https://pytorch.org/tutorials/beginner/transformer_tutorial.html, 2022a.

Pytorch. Imagenet training in pytorch. https://github.com/pytorch/examples/tree/main/imagenet, 2022b.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. art. arXiv:1910.10683, October 2019. doi: 10.48550/arXiv.1910.10683.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL http://jmlr.org/papers/v21/20-074.html.

Rosenfeld, J. S., Rosenfeld, A., Belinkov, Y., and Shavit, N. A Constructive Prediction of the Generalization Error Across Scales. art. arXiv:1909.12673, September 2019. doi: 10.48550/arXiv.1909.12673.

Schneider, F., Nado, Z., Agarwal, N., Dahl, G. E., and Hennig, P. HITY workshop poll, NeurIPS 2022. https://github.com/fsschneider/HITYWorkshopPoll, 2022.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms. art. arXiv:1707.06347, July 2017. doi: 10.48550/arXiv.1707.06347.

Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., and de Freitas, N. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104 (1):148–175, 2016. doi: 10.1109/JPROC.2015.2494218.

Shaw, P., Uszkoreit, J., and Vaswani, A. Self-attention with relative position representations. *arXiv preprint arXiv:1803.02155*, 2018.

Snoek, J., Larochelle, H., and Adams, R. P. Practical Bayesian Optimization of Machine Learning Algorithms. art. arXiv:1206.2944, June 2012. doi: 10.48550/arXiv.1206.2944.

Swersky, K., Snoek, J., and Adams, R. P. Multi-task bayesian optimization. In Burges, C., Bottou, L., Welling, M., Ghahramani, Z., and Weinberger, K. (eds.), *Advances in Neural Information Processing Systems*, volume 26, 2013. URL https://proceedings.neurips.cc/paper/2013/file/f33ba15effa5c10e873bf3842afb46a6-Paper.pdf.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2818–2826, 2016.

Tan, M. and Le, Q. V. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. art. arXiv:1905.11946, May 2019. doi: 10.48550/arXiv.1905.11946.

Tay, Y., Dehghani, M., Rao, J., Fedus, W., Abnar, S., Chung, H. W., Narang, S., Yogatama, D., Vaswani, A., and Metzler, D. Scale efficiently: Insights from pre-training and fine-tuning transformers. *CoRR*, abs/2109.10686, 2021. URL https://arxiv.org/abs/2109.10686.

Turner, R., Eriksson, D., McCourt, M., Kiili, J., Laaksonen, E., Xu, Z., and Guyon, I. Bayesian optimization is superior to random search for machine learning hyperparameter tuning: Analysis of the black-box optimization challenge 2020. *CoRR*, abs/2104.10201, 2021. URL https://arxiv.org/abs/2104.10201.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Wang, C., Wu, Q., Huang, S., and Saied, A. Economical hyperparameter optimization with blended search strategy. In *The Ninth International Conference on Learning Representations (ICLR 2021)*, May 2021.

Wang, Z., Hutter, F., Zoghi, M., Matheson, D., and de Freitas, N. Bayesian Optimization in a Billion Dimensions via Random Embeddings. art. arXiv:1301.1942, January 2013. doi: 10.48550/arXiv.1301.1942.

Weng, J., Lin, M., Huang, S., Liu, B., Makoviichuk, D., Makoviychuk, V., Liu, Z., Song, Y., Luo, T., Jiang, Y., Xu, Z., and Yan, S. EnvPool: A highly parallel reinforcement learning environment execution engine. *arXiv preprint arXiv:2206.10558*, 2022.

Wu, J., Toscano-Palmerin, S., Frazier, P. I., and Wilson, A. G. Practical Multi-fidelity Bayesian Optimization for Hyperparameter Tuning. art. arXiv:1903.04703, March 2019. doi: 10.48550/arXiv.1903.04703.

Yang, G., Hu, E. J., Babuschkin, I., Sidor, S., Liu, X., Farhi, D., Ryder, N., Pachocki, J., Chen, W., and Gao, J. Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer. art. arXiv:2203.03466, March 2022. doi: 10.48550/arXiv.2203.03466.

Zhai, X., Kolesnikov, A., Houlsby, N., and Beyer, L. Scaling vision transformers. *CoRR*, abs/2106.04560, 2021. URL https://arxiv.org/abs/2106.04560.

A. CARBS Algorithm

Algorithm 1 Simplified CARBS Algorithm

Input: search center μ, search radius σ_{search}, number of candidates per pareto front point N_{cand}
Add observations to X_i
repeat
  Sample suggestions from $\mathcal{N}(\mu, \sigma_{\text{search}})$.
until X_i has observations
repeat
  Calculate the pareto front PF from X_i using Eq. (1)
  Sample N_{cand} candidate points X_{cand} from Gaussian distributions of radius σ_{search} around each point in PF
  Fit Gaussian process models $\mathcal{GP}_y$, $\mathcal{GP}_c$, and $\mathcal{GP}_{\text{pf}}$ with observations X_i and PF according to Eq. (3)
  for each candidate point **do**
    Calculate expected costs $\tilde{c}$, outputs $\tilde{y}$, and pareto front values $\tilde{y}_{\text{pf}}$
    Evaluate the acquistion function $\alpha_{\text{EI}-\text{pf}}$ using Eq. (4)
  end for
  Return candidate point with highest acquisition function value
until terminated

B. ProcGen Experiment

B.1. Search parameters

name	Space type	Search center	min	max
total env steps	Log	2e+07	1e+06	∞
num steps	Log	256	1	1024
num workers	Log	256	1	1024
minibatch size	Log	8192	1	8192
ppo epochs	Linear	3	1	∞
lr	Log	0.0005	0	∞
clip range	Logit	0.2	0	1
lam	Logit	0.95	0	1
discount	Logit	0.999	0	1
entropy coef	Log	0.01	0	∞
value loss coef	Log	0.5	0	∞
cnn base width	Log	16	1	∞
mlp width	Log	256	1	∞

Table 1. Search parameters for tuning PPO on ProcGen.

B.2. Per-task scores

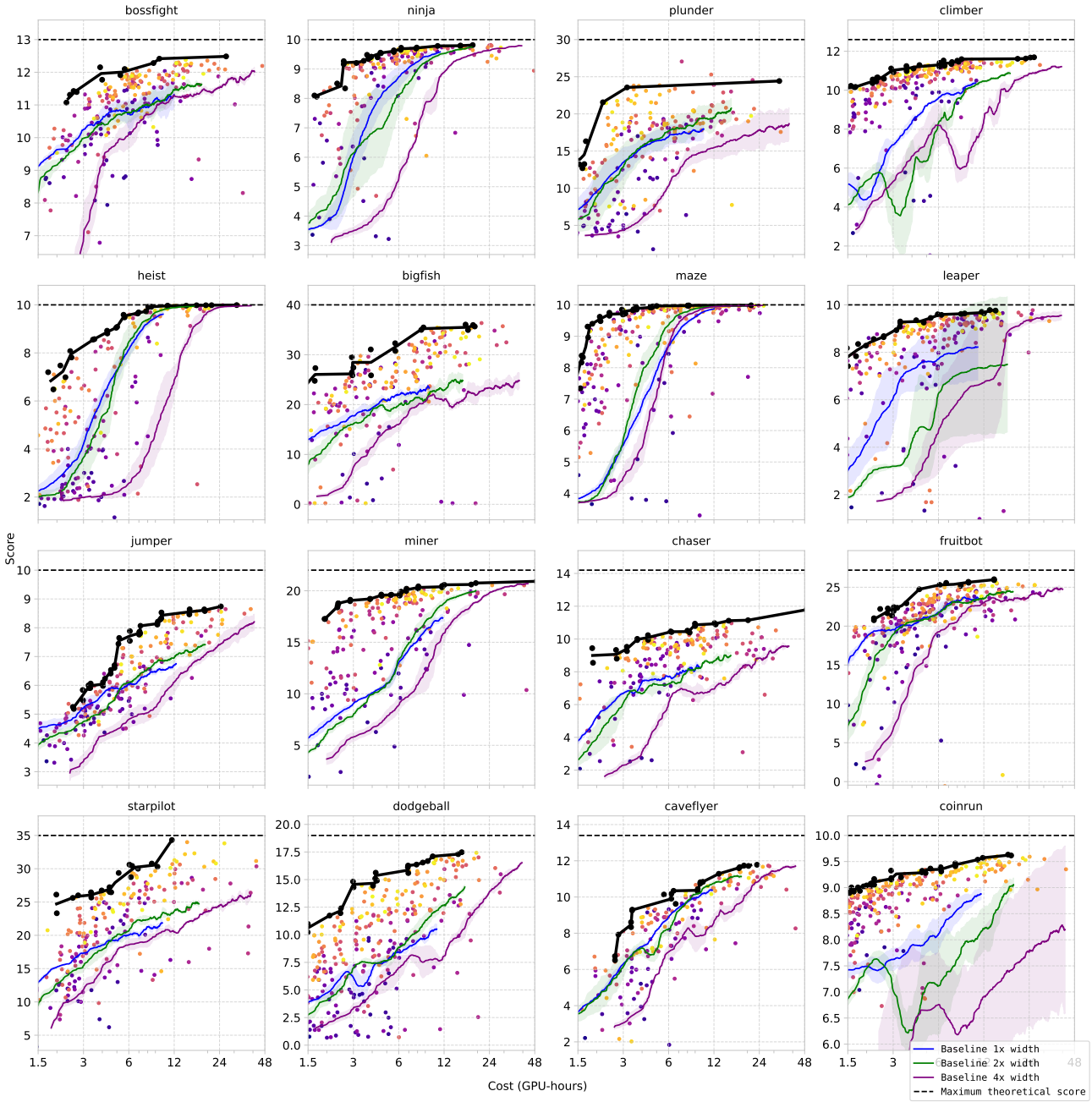

Figure 7. Performance-cost Pareto curves on each ProcGen task after tuning PPO with CARBS.

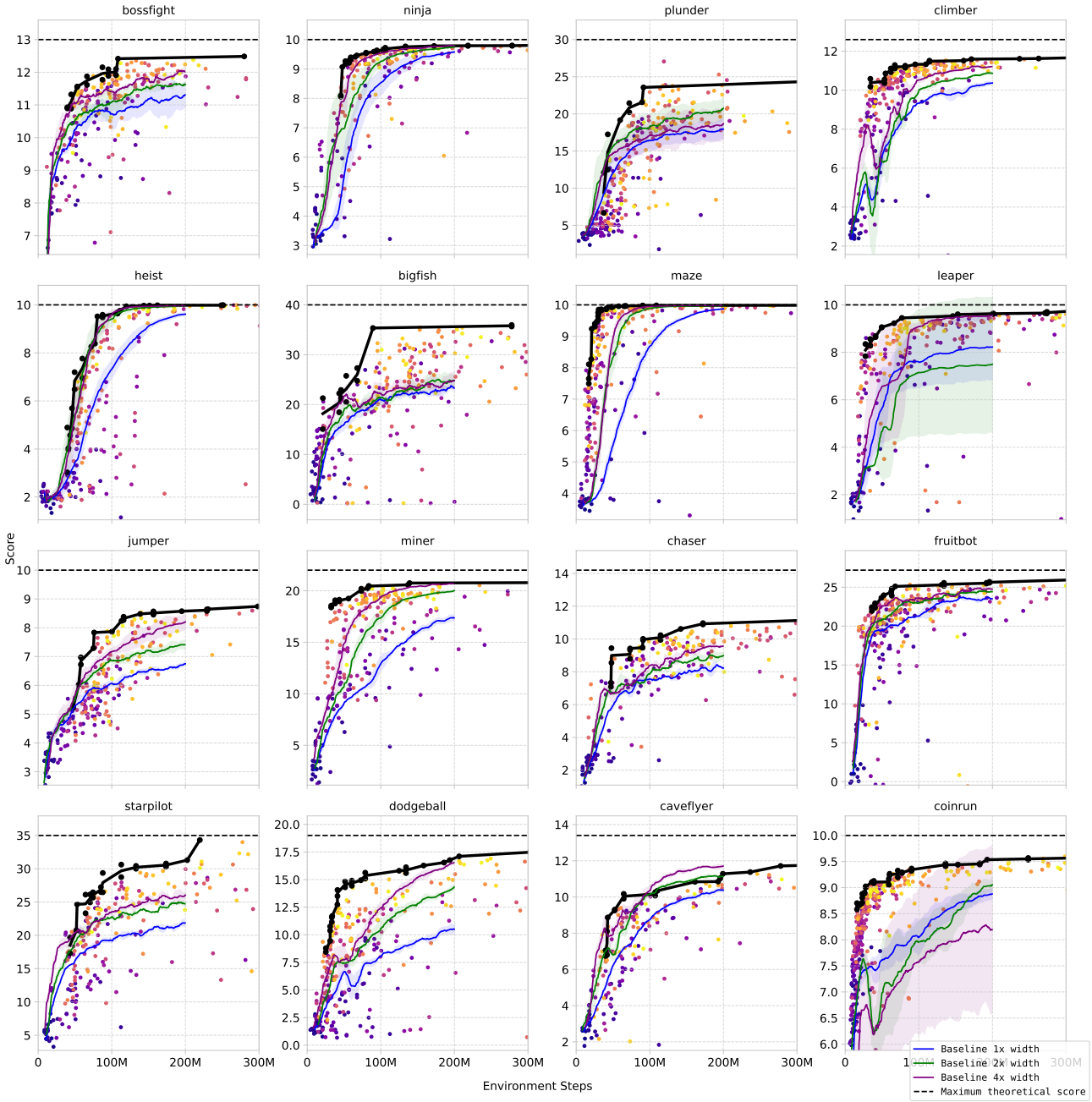

Figure 8. Performance-sample count Pareto curves on each ProcGen task after tuning PPO with CARBS.

C. GPT Tuning and Scaling Laws

C.1. Isoflop curve method

We set out to replicate the results of the Chinchilla paper by taking isoflop slices from our pool of observations. This is a bit tricky, as the observations we made vary along 19 different dimensions. We approximate the number of flops as 6ND, where N is the number of model parameters and D is the number of tokens

To get each isoflop curve, we first select observations near the target flop count (from 0.8x to 1.2x). We fix most of those input parameters, and only adjust the token count such to match the target flop count. We then pass those flop-adjusted parameters to our GP model to get a target-flop corrected output, as well as error bounds from the model. Finally, to reduce the effect of models that are poorly tuned in other dimensions, we split the observations into k bins by their parameter count, and choose the best output for each bin. We fit each curve in the manner of Chinchilla, and plot each of these isoflop curves in Figure 3.

C.2. Scaling for all parameters

name	Space type	Search center	min	max
learning rate	Log	0.0004	0	∞
num train tokens	Log	2.5e+09	1e+08	∞
num layers	Log	12	2	∞
num heads	Log	12	1	∞
kv size	Log	64	8	128
ffw size	Log	3072	8	∞
microbatch size	Log	12	1	∞
max seq len	Log	1024	2	∞
init std	Log	1	0	∞
embedding init std	Log	1	0	∞
output scaling	Log	1	0	∞
attn pdrop	Logit	0.01	0.001	0.5
resid pdrop	Logit	0.01	0.001	0.5
emb pdrop	Logit	0.01	0.001	0.5
weight decay	Log	0.1	0	∞
grad clip norm	Log	1	0	∞
alpha f	Logit	0.1	0	1
warmup frac	Logit	0.02	0	1
attn softmax scale	Log	8	0	∞

Table 2. Search parameters for tuning GPT on the C4 dataset.

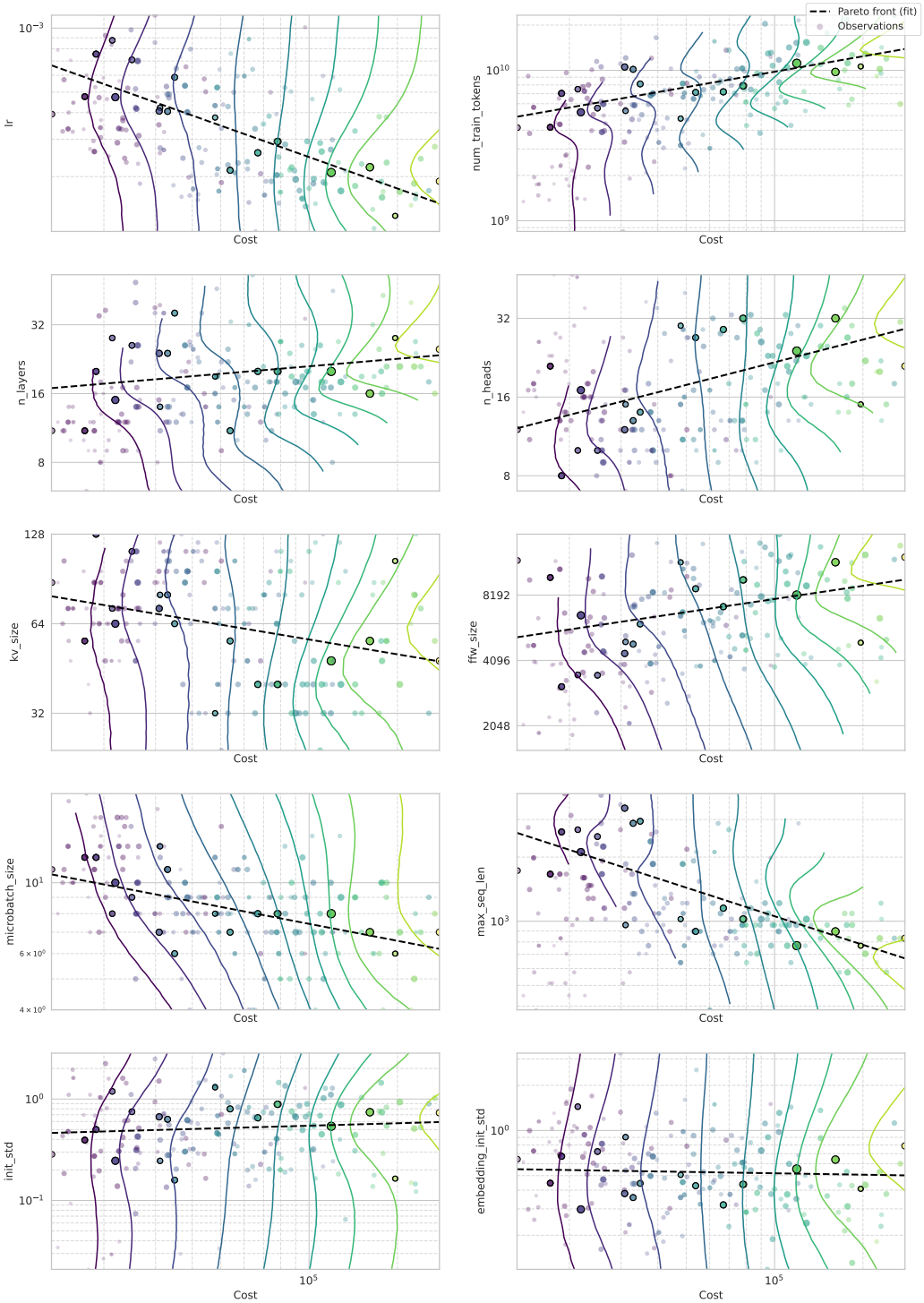

Figure 9. Fit pareto front for first half of GPT parameters tuned on the C4 dataset with CARBS. Colored lines indicate loss contours of the GP model. See other image for colorbar.

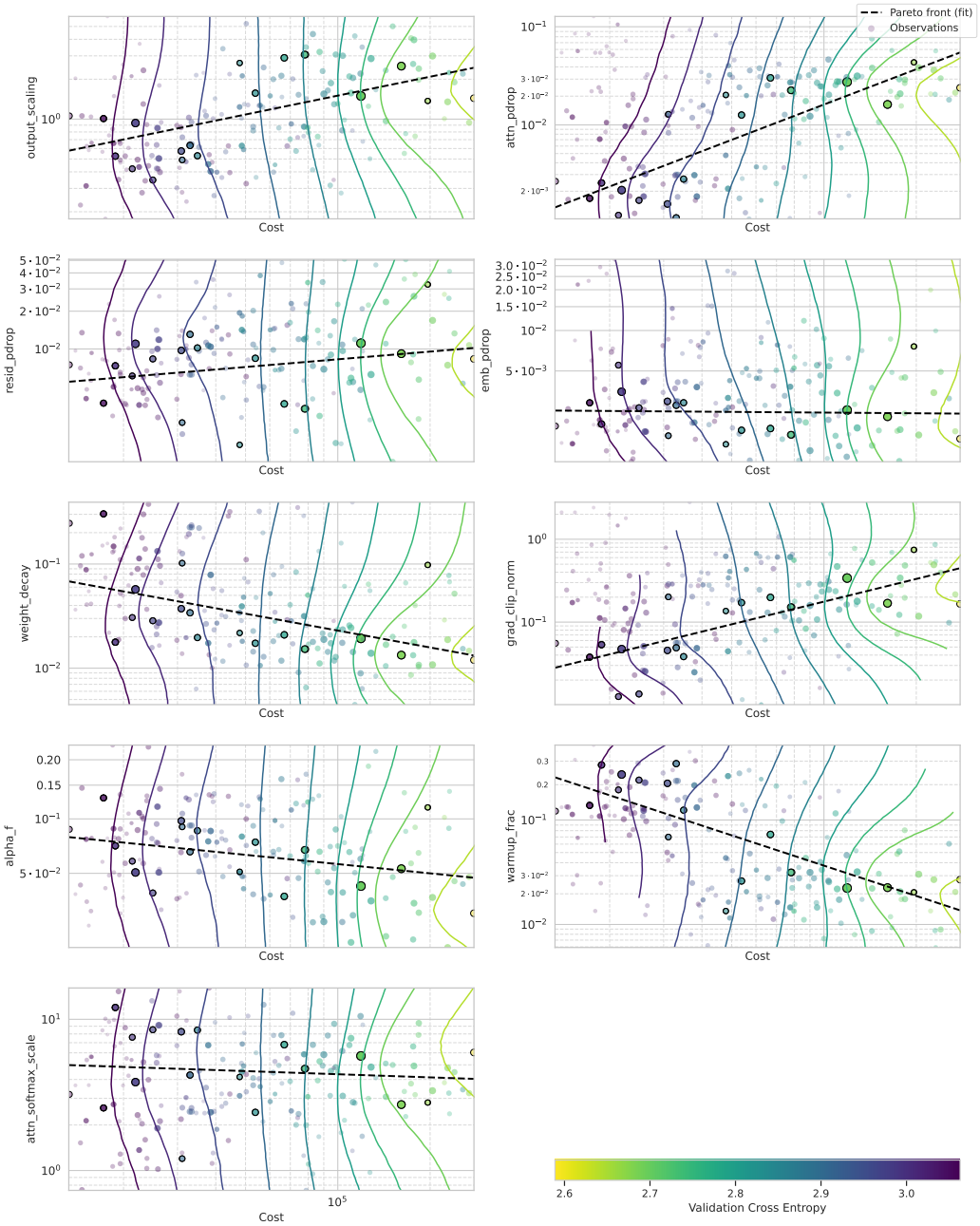

Figure 10. Fit pareto front for second half of GPT parameters tuned on the C4 dataset with CARBS. Colored lines indicate loss contours of the GP model.

D. Search Parameters For Comparison Experiments

name	Space type	Search center	CARBS min - max	Other min - max
total timesteps	Log	2e+06	2e+05 - ∞	5e+05 - 2e+08
learning rate	Log	0.0003	0 - ∞	1e-05 - 0.01
update epochs	Log	10	1 - ∞	2 - 32
max grad norm	Log	0.5	0 - ∞	0.01 - 5
num steps	Log	32	4 - ∞	8 - 2048
num envs	Log	64	4 - ∞	16 - 2048
num minibatches	Log	32	1 - ∞	2 - 128
clip coef	Logit	0.2	0 - 1	0.01 - 0.9
gae lambda	Logit	0.95	0 - 1	0.5 - 0.999
ent coef	Log	0.001	0 - ∞	1e-05 - 0.1
gamma	Logit	0.99	0 - 1	0.9 - 0.999

Table 3. Search parameters for reinforcement learning task. "Other" search bounds are used for algorithms other than CARBS.

name	Space type	Search center	CARBS min - max	Other min - max
kv dim	Log	32	2 - ∞	8 - 128
nhead	Log	4	1 - ∞	2 - 32
nhid	Log	512	4 - ∞	256 - 5120
nlayers	Log	4	1 - ∞	1 - 10
dropout	Logit	0.1	0 - 1	5e-03 - 0.5
lr	Log	0.0003	0 - ∞	1e-04 - 0.01
clip grad norm	Log	1	0 - ∞	0.01 - 10
train tokens	Log	1e+08	1e+06 - ∞	1e+07 - 1e+09
batch size	Log	64	2 - ∞	16 - 256
bptt	Log	128	2 - ∞	32 - 1024

Table 4. Search parameters for language modeling task. "Other" search bounds are used for algorithms other than CARBS.

name	Space type	Search center	CARBS min - max	Other min - max
epochs	Log	45	5 - ∞	12 - 1000
lr	Log	0.1	0 - ∞	0.01 - 1
momentum	Logit	0.9	0 - 1	0.5 - 0.99
weight decay	Log	0.0001	0 - ∞	1e-06 - 0.01
label smoothing	Logit	0.1	1e-03 - 1	1e-03 - 0.5
batch size	Log	256	16 - ∞	32 - 512
image dim	Log	160	64 - ∞	64 - 400
num blocks	Log	16	8 - 139	8 - 64
width per group	Log	64	8 - 512	16 - 200
rand augment magnitude	Log	10	2 - 24	2 - 24

Table 5. Search parameters for image classification task. "Other" search bounds are used for algorithms other than CARBS.